Use Cases

for

Continuity

Version 1.0 approved

Prepared by Jerral Chua Fung Yong, Jethani Samarth, Sinha Armaan, Faye Chan, Lai Yi, Jain Sparsh, Kataruka Mohika

College of Computing and Data Science, Nanyang Technological University

30th August 2026

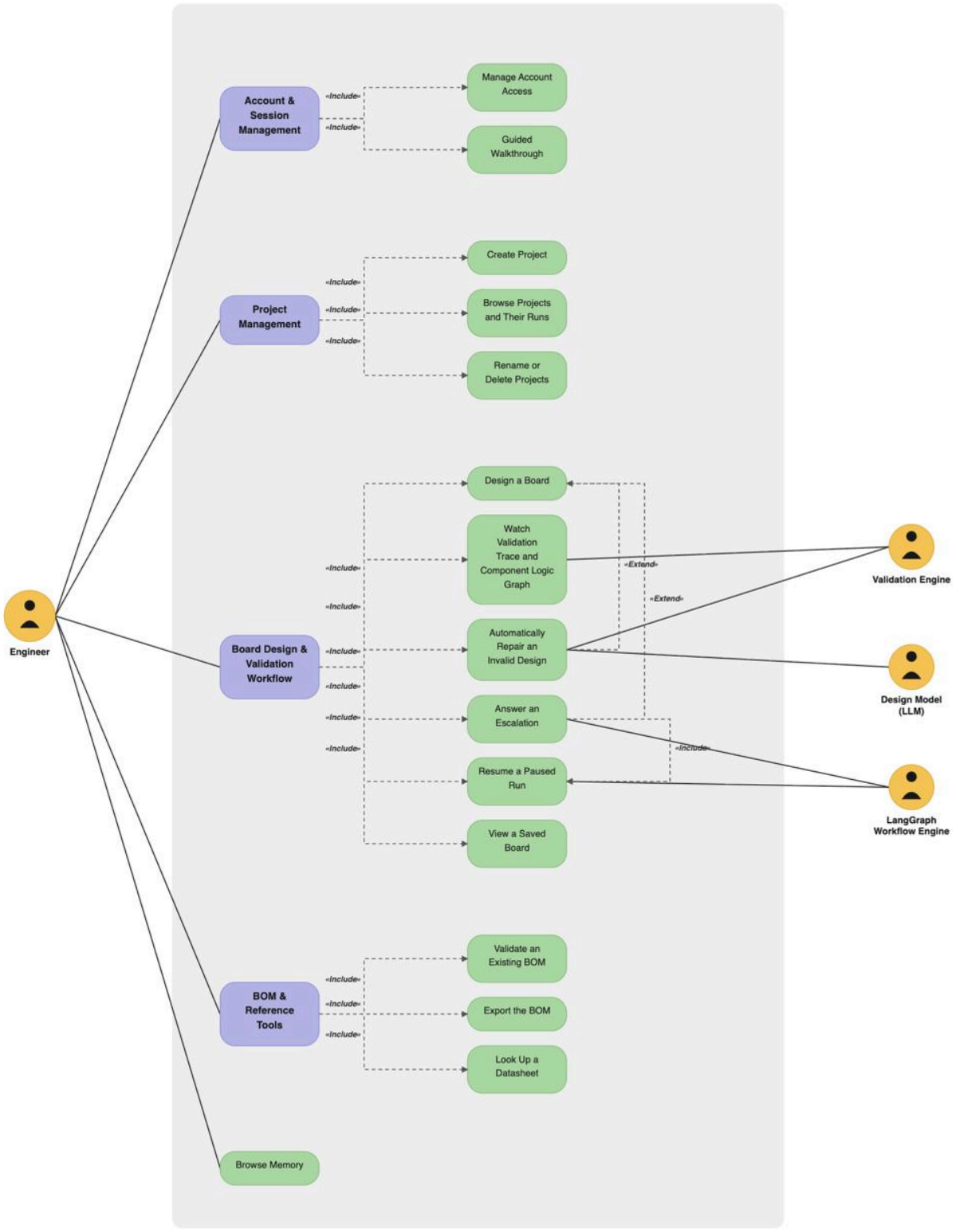
Revision History

Name	Date	Reason For Changes	Version
Jerral, Samarth, Armaan, Faye, Lai Yi	30 Aug - 1 Sep 2026	First Version	1.0

Copyright © 2004 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Use Case List

Primary Actor	***Use Cases***
Engineer	UC-1: Manage Account Access
Engineer	UC-2: Create Project
Engineer	UC-3: Browse projects
Engineer	UC-4: Rename or Delete projects
Engineer	UC-5: Guided Walkthrough
Engineer	UC-6: Design a Board
Engineer	UC-7: Watch Validation Trace and Component Logic Graph
Engineer	UC-8: Automatically Repair an Invalid Design
Engineer	UC-9: Answer an Escalation
Engineer	UC-10: Resume a Paused Run
Engineer	UC-11: View a Saved Board
Engineer	UC-12: Validate an Existing BOM
Engineer	UC-13: Export the BOM
Engineer	UC-14: Look Up a Datasheet
Engineer	UC-15: Browse Memory


Engineer
Account & Session Management
Project Management
Board Design & Validation Workflow
BOM & Reference Tools
Browse Memory
Manage Account Access
Guided Walkthrough
Create Project
Browse Projects and Their Runs
Rename or Delete Projects
Design a Board
Watch Validation Trace and Component Logic Graph
Automatically Repair an Invalid Design
Answer an Escalation
Resume a Paused Run
View a Saved Board
Validate an Existing BOM
Export the BOM
Look Up a Datasheet
«Include»
«Extend»
Validation Engine
Design Model (LLM)
LangGraph Workflow Engine

Use Case ID:	UC-1		
Use Case Name:	Manage Account Access		
Created By:	Faye Chan	Last Updated By:	
Date Created:	30-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an Engineer to obtain and manage access to the system. A new Engineer can register for an account while an existing Engineer can sign in. The system can determine the currently authenticated Engineer through the current-user endpoint and an authenticated Engineer can sign out when finished. Relevant routes: ● POST /auth/register ● POST /auth/login ● POST /auth/logout ● GET /auth/me
Trigger:	The use case begins when the Engineer chooses to register, sign in or sign out or when the application needs to determine whether the Engineer already has an authenticated session.
Preconditions:	1. The system is running and the authentication API is available. 2. For registration, the Engineer does not already have an authenticated session. 3. For signing in, the Engineer already has an account. 4. For sign out, the Engineer currently has an authenticated session.
Postconditions:	1. After successful registration, the Engineer has an account recognised by the system. 2. After successful sign-in, an authenticated session exists for the Engineer. 3. After a successful GET /auth/me, the application knows the identity/authentication state of the current Engineer. 4. After successful sign out, the Engineer's authenticated session is ended. 5. If authentication fails, the Engineer remains unauthenticated and no valid authenticated session is created.
Normal Flow:	**UC-1.0 Register Account** 1. The Engineer opens the account registration interface. 2. The system displays the required registration fields. 3. The Engineer enters the required account information. 4. The Engineer submits the registration request. 5. The client sends the registration information to POST /auth/register. 6. The system validates the information and creates the new Engineer account. 7. The system establishes or returns the authentication state required by the application. 8. Does the app calls GET /auth/me to obtain info and log in automatically or does it go back to the sign in page
Alternative Flows:	**UC-1.1 Sign-in to Existing Account**

	1. The Engineer chooses to sign-in instead of register. 2. The system displays the sign-in interface. 3. The Engineer enters the required authentication information. 4. The Engineer submits the sign-in request. 5. The client sends the request to POST /auth/login 6. The system validates the credentials and establishes an authenticated session. 7. The application calls GET /auth/me to retrieve the current Engineer. 8. The system returns the current Engineer's account information. 9. The application displays the authenticated workspace. **UC-1.2 Restore or Check Existing Session** 1. The Engineer opens or refreshes the application. 2. The application sends a request to GET /auth/me. 3. The system checks the existing authentication state. 4. If the session is valid, the system returns information for the currently authenticated Engineer. 5. The application restores the Engineer's authenticated state. 6. The Engineer continues using the application. **UC-1.3 Sign-out** 1. An authenticated Engineer chooses to sign out. 2. The application sends a request to POST /auth/logout. 3. The system ends the Engineer's current session. 4. The application clears its authenticated user state. 5. The application returns the Engineer to an unauthenticated screen.
Exceptions:	**UC-1.0.E.1 Invalid Registration Information** 1. The registration request does not satisfy the server's validation requirements. 2. The system rejects the request. 3. No account is created from the failed request. 4. The application informs the Engineer that registration could not be completed. **UC-1.0.E.2 Registration Cannot Be Completed** 1. The system is unable to process the registration request. 2. The system returns an error. 3. The account is not treated as successfully created. 4. The application informs the Engineer that registration failed. **UC-1.1.E.1 Authentication Fails** 1. The information given to POST /auth/login is not accepted. 2. The system rejects the authentication attempt. 3. No authenticated session is established. 4. The application informs the Engineer that sign-in was unsuccessful. **UC-1.2.E.1 No Valid Current Session**

	1. GET /auth/me determines that there is no valid authenticated Engineer. 2. The application treats the Engineer as unauthenticated. **UC-1.3.E.1 Logout Request Fails** 1. The logout request cannot be completed normally. 2. The application must not show that server-side logout has succeeded. 3. The Engineer is informed or the application handles the failed request according to its authentication error handling.
Includes:	None
Priority:	High
Frequency of Use:	Registration: once per Engineer account Sign-in / session restoration: regularly Sign out: as required when Engineer finishes a session
Business Rules:	Only an authenticated Engineer may access functionality that requires authenticated access.
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

Use Case ID:	UC-2		
Use Case Name:	Create Project		
Created By:	Faye Chan	Last Updated By:	
Date Created:	30-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an authenticated Engineer to create a new project in the workspace. The Engineer provides a prompt required by the interface and the system creates and stores the project using POST /projects.
Trigger:	The Engineer chooses the option to create a new project.
Preconditions:	1. The Engineer has successfully authenticated with the system. 2. The project API is available.
Postconditions:	1. A new project is successfully stored by the system. 2. The project is available to the Engineer through the interface. 3. The system returns sufficient project information for the Engineer to identify and display the newly created project. 4. If creation fails, a successfully created project must not be shown to the Engineer.
Normal Flow:	**UC-2.0 Create Project** 1. The Engineer selects the option to create a project. 2. The system displays the project creation interface. 3. The Engineer submits the prompt required by the interface. 4. The client sends the project creation request to POST /projects. 5. The system validates the request, creates and stores the project. 6. The system returns the newly created project's information. 7. The application displays the new project or updates the project interface.
Alternative Flows:	**UC-2.1 Cancel Project Creation** 1. Before submitting the request, the Engineer leaves or cancels the creation process. 2. No POST /projects request is completed for the abandoned project. 3. No new project is created.
Exceptions:	**UC-2.0.E.1 Invalid Project Information** 1. The project request fails server or client side validation. 2. The system does not create a valid project. 3. The application informs the Engineer that the supplied information cannot be accepted. **UC-2.0.E.2 Engineer not Authenticated** 1. The system determines that the Engineer does not have valid authentication. 2. The project creation request is rejected. 3. No project is created. **UC-2.0.E.3 Project Creation Fails** 1. The server cannot complete POST /projects and returns an error.

	2. The application does not treat the project as successfully created.
Includes:	None
Priority:	High
Frequency of Use:	Performed whenever an Engineer starts a new project. Frequency depends on the number of projects created by each Engineer.
Business Rules:	A project may only be created by an authenticated Engineer.
Special Requirements:	The operation must provide clear feedback whether project creation succeeded or failed.
Assumptions:	None
Notes and Issues:	None

Use Case ID:	UC-3		
Use Case Name:	Browse Projects and Their Runs		
Created By:	Jerral	Last Updated By:	
Date Created:	31-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case lets a signed-in engineer see the projects they own and see the runs inside a project they pick. The app gets the project list from GET /projects and gets the runs for one project from GET /projects/{id}/threads. This use case only reads data. It does not change anything. Relevant routes: ● GET /projects ● GET /projects/{id}/threads
Trigger:	The engineer opens the projects page or picks a project to see the runs inside it
Preconditions:	1. The engineer is signed in. 2. The project API is working.
Postconditions:	1. The engineer sees the projects the system has for them. 2. For the project they picked, the engineer sees the runs inside it. 3. No project, run, or other data is created, changed, or deleted. 4. If a list fails to load, the app does not show it as a full list.
Normal Flow:	**UC-3.0 Browse Projects and Their Runs** 1. The engineer opens the projects page. 2. The app sends a request to GET /projects. 3. The system sends back the projects that belong to the engineer. 4. The app shows the project list. 5. The engineer picks a project. 6. The app sends a request to GET /projects/{id}/threads. 7. The system sends back the runs inside that project. 8. The app shows the runs, with enough detail for the engineer to tell them apart.
Alternative Flows:	**UC-3.1 The Engineer Has No Projects** 1. GET /projects sends back no projects. 2. The app shows an empty list instead of an error. 3. The app offers the engineer the option to create a project. **UC-3.2 The Project Has No Runs** 1. GET /projects/{id}/threads sends back no runs. 2. The app shows that the project has no runs yet, instead of showing an error. **UC-3.3 Load the Lists Again** 1. The engineer comes back to the projects page after creating, renaming, or deleting a project. 2. The app asks for the lists again. 3. The app shows what the system sends back.
Exceptions:	**UC-3.0.E.1 The Project List Fails to Load** 1. GET /projects fails, and the system sends back an error.

	2. The app does not show an empty list as if it were the real answer. 3. The app tells the engineer the projects could not be loaded. **UC-3.0.E.2 The Run List Fails to Load** 1. GET /projects/{id}/threads fails, and the system sends back an error. 2. The app does not show an empty run list as if it were the real answer. 3. The app tells the engineer the runs could not be loaded.
Includes:	None
Priority:	High
Frequency of Use:	Often. Used at the start of most sessions, and every time the Engineer moves between projects.
Business Rules:	An engineer can only see their own projects. Looking at the lists does not change any saved data.
Special Requirements:	A failed load must look different from an empty list so the engineer is not misled.
Assumptions:	Each run belongs to one project only.
Notes and Issues:	Not yet decided: whether the lists need paging, sorting, or searching once an engineer has many projects. The frontend and backend developers will decide this before the interface contract is frozen.

Use Case ID:	UC-4		
Use Case Name:	Rename or Delete a Project		
Created By:	Jerral	Last Updated By:	
Date Created:	31-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an authenticated engineer to change the name of a project they hold or to remove a project from the system. Renaming is performed through PATCH /projects/{id} and removal through DELETE /projects/{id}. Both operations change stored state and must report clearly whether they succeeded. Relevant routes: ● PATCH /projects/{id} ● DELETE /projects/{id}
Trigger:	The engineer chooses to rename or to delete a project from the project list or the project workspace.
Preconditions:	1. The engineer has successfully authenticated with the system. 2. The project already exists and is held for this engineer. 3. The project API is available.
Postconditions:	1. After a successful rename, the system holds the project under the new name and returns it. 2. After a successful deletion, the project is no longer returned to the engineer by the system. 3. If a rename fails, the stored name is unchanged and the new name is not shown to the engineer as saved. 4. If a deletion fails, the project is not shown to the engineer as deleted.
Normal Flow:	**UC-4.0 Rename a Project** 1. The engineer selects the option to rename a project. 2. The application displays the project's current name for editing. 3. The engineer enters the new name. 4. The engineer submits the change. 5. The client sends the new name to PATCH /projects/{id}. 6. The system validates the request and updates the stored project. 7. The system returns the updated project information. 8. The application displays the project under its new name.
Alternative Flows:	**UC-4.1 Delete a Project** 1. The engineer selects the option to delete a project. 2. The application asks the engineer to confirm the deletion. 3. The engineer confirms. 4. The client sends a request to DELETE /projects/{id}. 5. The system removes the project. 6. The system confirms the removal. 7. The application removes the project from the displayed list. **UC-4.2 Cancel a Rename** 1. Before submitting, the engineer cancels or leaves the rename. 2. No PATCH /projects/{id} request is completed. 3. The stored name is unchanged.

	UC-4.3 Decline a Deletion 1. The engineer does not confirm the deletion when asked. 2. No DELETE /projects/{id} request is sent. 3. The project is retained.
Exceptions:	**UC-4.0.E.1 Invalid Project Name** 1. The new name fails client- or server-side validation. 2. The system does not update the project. 3. The application informs the engineer that the name cannot be accepted. **UC-4.0.E.3 Rename Cannot Be Completed** 1. The server cannot complete PATCH /projects/{id} and returns an error. 2. The application does not treat the rename as saved. 3. The application returns the displayed name to the last name the system confirmed and informs the engineer. **UC-4.0.E.3 Deletion Cannot Be Completed** 1. The server cannot complete DELETE /projects/{id} and returns an error. 2. The application does not treat the project as deleted. 3. The project remains in the displayed list, and the engineer is informed. **UC-4.0.E.4 Project Already Absent** 1. The system reports that the project does not exist or is not held for this engineer. 2. The application does not report a successful deletion it did not perform. 3. The application brings its displayed list back in line with the system.
Includes:	None
Priority:	Medium
Frequency of Use:	Occasionally. Renaming is performed as an engineer reorganises their work; deletion when a project is no longer needed.
Business Rules:	Only an authenticated engineer may rename or delete a project, and only a project held for their own account. Deletion is a durable change and must be confirmed by the engineer before it is sent.
Special Requirements:	The operation must state clearly whether it succeeded or failed. The interface must not display a renamed or deleted state that the system has not confirmed.
Assumptions:	Renaming changes the project's name only and does not affect the runs recorded against it.
Notes and Issues:	Whether deleting a project also removes the runs recorded against it and anything the system retained from those runs is unresolved. To be decided by the backend developer before implementation.

Use Case ID:	UC-5		
Use Case Name:	Guided Walkthrough		
Created By:	Faye Chan	Last Updated By:	
Date Created:	30-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case introduces a new Engineer to the application's design workflow through a guided walkthrough. The walkthrough is implemented by walkthrough.tsx and uses the demo-design endpoint POST /design/demo.
Trigger:	The use case begins when a new account enters the application's guided walkthrough experience.
Preconditions:	1. The Engineer has successfully created and signed in to a new account. 2. The walkthrough interface and POST /design/demo is available.
Postconditions:	1. The Engineer has been presented with the application's guided introductory workflow. 2. On successful completion, the Engineer can proceed from onboarding to normal application use.
Normal Flow:	**UC-5.0 Complete Guided Walkthrough** 1. A new Engineer enters the application after account access has been established. 2. The system displays the guided walkthrough implemented by walkthrough.tsx. 3. The Engineer follows the walkthrough instructions. 4. The Engineer completes the walkthrough and proceeds to normal application use.
Alternative Flows:	None
Exceptions:	**UC-5.0.E.1 Walkthrough Cannot Be Loaded** 1. The walkthrough interface cannot be displayed or initialised correctly. 2. The system prevents the failure from incorrectly marking the walkthrough as completed. 3. The application reports or handles the failure according to its normal UI error handling.
Includes:	None
Priority:	Medium
Frequency of Use:	For new account only unless a mechanism is provided to replay it
Business Rules:	None
Special Requirements:	None
Assumptions:	None
Notes and Issues:	None

Use Case ID:	UC-6		
Use Case Name:	Design a Board from a Plain-Language Brief		
Created By:	Jerral	Last Updated By:	
Date Created:	31-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an authenticated engineer to describe a circuit board in plain language and receive a bill of materials made of real parts, with every rule checked against every part. The engineer submits the brief through POST /design, and the system streams the progress of the run to the application over server-sent events. The system plans the component slots and power rails, sources a real part for each slot, re-checks every rule after every placement, repairs the conflicts it can, and finishes with a bill of materials in which every verdict carries the field, the value, and the source it was read from. Relevant routes: ● POST /design (response streamed over SSE)
Trigger:	The engineer submits a plain-language brief describing a board.
Preconditions:	1. The engineer has successfully authenticated with the system. 2. A project exists for the run to be recorded against. 3. The design API and its event stream are available. 4. The engineer has entered a brief.
Postconditions:	1. A run is recorded against the project and can be found again through UC-3. 2. On a completed run, the engineer is shown a bill of materials in which every part carries the verdicts produced for it. 3. Every verdict shown carries the field name, the value read, and the source it came from. 4. A rule that could not be evaluated is shown as unchecked, naming the missing field, and is never shown as a pass. 5. A conflict is shown together with every rule that ran on the failing part, including the rules that passed. 6. If the run does not complete, no partial result is presented to the Engineer as a validated bill of materials.
Normal Flow:	**UC-6.0 Design a Board from a Brief** 1. The engineer opens the design workspace for a project. 2. The system displays the brief input. 3. The engineer describes the board in plain language. 4. The engineer submits the brief. 5. The client sends the brief to POST /design and begins consuming the event stream. 6. The system accepts the brief, starts the run, and streams its progress in order. 7. The system converts the brief into component slots and power rails and streams them to the application. 8. For each slot, the system searches the distributor sources, selects a real part, and streams that part together with where its values came from.

	9. After each placement, the engine reruns every rule against every part on the board. 10. The system streams the verdicts, including the ones that pass and the ones that could not be evaluated. 11. Where a rule fails, the engine determines which parts may be changed, the model selects one move from that set, the engine applies it, and it re-runs every rule on the result. 12. When every slot is filled and every rule has been run, the system finalizes the bill of materials with the price and stock of each part. 13. The system streams the completion of the run and closes the stream. 14. The application displays the board, its verdicts, and the bill of materials.
Alternative Flows:	**UC-6.1 Run Completes With an Unresolved Conflict** 1. A conflict cannot be repaired, either because no change is available or because the repair limit for that slot has been reached. 2. The system raises the conflict to the engineer as an explicit trade-off rather than repairing it further. 3. The run completes. 4. The application shows the conflict as unresolved and does not show the affected part as passing. UC-**6.2 A Rule Cannot Be Evaluated** 1. A value a rule needs is not published by any source. 2. The engine reports the rule as unchecked and names the missing field. 3. No default value is substituted for the missing figure. 4. The run continues, and the verdict is shown to the engineer as unchecked.
Exceptions:	**UC-6.0.E.1 Brief Not Accepted** 1. The brief is empty or fails validation. 2. No run is started. 3. The application informs the engineer that the brief failed. **UC-6.0.E.2 Event Stream Interrupted** 1. The stream is broken before the run reports completion. 2. The application does not present the part of the board it received as a finished, validated bill of materials. 3. The application reports that the run is incomplete. **UC-6.0.E.3 A Parts Source Is Unavailable** 1. A distributor source cannot be reached during the run. 2. The system does not fill the affected slot with an unverified part and does not emit a verdict without a source. 3. The affected slots are reported to the engineer as unresolved.

	UC-6.0.E.4 Run Fails Before Completion 1. The run cannot be completed for an unanticipated reason. 2. The run is left in an explicitly incomplete state and is not recorded as a finished design. 3. No part of the board is reported as having passed validation.
Includes:	None
Priority:	High
Frequency of Use:	This is the system's central operation. Performed each time an engineer designs or redesigns a board, several times per project.
Business Rules:	1. Only an authenticated engineer may start a design run. 2. No pass or fail verdict may be produced by the language model. 3. A value that is not published is reported as unchecked and is never replaced by a default. 4. Every verdict must carry the field name, the value read, and the source. 5. Every rule is re-run against every part after every placement. 6. A component slot is repaired no more than three times before the conflict is raised to the engineer.
Special Requirements:	1. The first event is rendered within 2.0 seconds of submission (DS-6). 2. A board of twelve slots or fewer completes within 130 seconds against live distributor sources (DS-5). 3. Streamed events arrive in order with no gaps and no duplicates (DS-6). 4. A conflict displays every rule that ran on the failing part, not only the rule that failed (DS-15).
Assumptions:	The brief is written in English. The distributor sources and the model provider are reachable for the duration of the run.
Notes and Issues:	1. Whether the run continues on the server after the engineer disconnects or is stopped is unresolved. To be decided by the backend developer. 2. Whether the engineer can cancel a run that is in progress is unresolved. To be decided with the frontend developer before the interface contract is frozen.

Use Case ID:	UC-7		
Use Case Name:	Watch Validation Trace and Component Logic Graph		
Created By:	Lai Yi	Last Updated By:	
Date Created:	1-9-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an authenticated Engineer to watch the validation trace and the component logic graph for a design run in the design workspace (design.tsx). It does not start a new design run (that is UC-6) and does not apply a repair (that is UC-8). The Engineer inspects how each rule was evaluated against each part, including passes, failures and unchecked rules, together with the graph of component relationships used in that evaluation. Relevant routes / screens: ● design.tsx (design workspace) ● streamed validation events from an already started design run (SSE from POST /design in UC-6, or stored verdicts for that run)
Trigger:	The Engineer opens the design workspace for a run that is producing or has already produced validation results, or the application begins rendering streamed validation events after a design run has started.
Preconditions:	1. The Engineer has successfully authenticated with the system. 2. A design run exists for the project and can be opened in the design workspace. 3. The design workspace (design.tsx) is available. 4. Validation events are being streamed, or stored verdicts already exist for the run.
Postconditions:	1. The Engineer can see the validation trace for the run. Each shown verdict carries the field name, the value read, and the source. 2. The Engineer can see the component logic graph for the current board. 3. Watching the trace and the graph does not create, change, or delete the board, the run, or any verdict. 4. A rule that could not be evaluated is shown as unchecked, naming the missing field, and is never shown as a pass. 5. If the trace or the graph cannot be loaded, the application does not present an empty view as if it were the real result.

Normal Flow:	**UC-7.0 Watch Validation Trace and Component Logic Graph** 1. The Engineer opens the design workspace for a project run (design.tsx). 2. The application displays the current board for that run. 3. The system provides the validation results for the run, either from the live event stream or from stored verdicts. 4. The application renders the validation trace: each rule, the part it ran on, the outcome (pass, fail, or unchecked), the field, the value, and the source. 5. The application renders the component logic graph showing the slots, the placed parts, and the relationships used in validation. 6. The Engineer inspects the trace and the graph. No change is submitted to the board. 7. If the run is still streaming, the application updates the trace and the graph as new events arrive, in order, with no gaps and no duplicates.
Alternative Flows:	**UC-7.1 Run Still in Progress** 1. The run has not yet reported completion. 2. The trace and the graph continue to update as events arrive. 3. The application does not present the board as a finished, validated bill of materials while the Engineer is only watching an in-progress run. **UC-7.2 Run Completed With Unresolved Conflicts** 1. The trace shows the failed rule together with every rule that ran on the failing part, including the rules that passed. 2. The component logic graph remains inspectable. 3. The application does not show the affected part as passing. **UC-7.3 A Rule Could Not Be Evaluated** 1. The trace shows the rule as unchecked and names the missing field. 2. The graph still displays the part. 3. No default value is substituted for the missing figure.

Exceptions:	**UC-7.0.E.1 Trace Cannot Be Loaded** 1. The system cannot provide the validation trace for the run. 2. The application informs the Engineer that the trace could not be loaded. 3. The application does not show an empty trace as if no rules had run. **UC-7.0.E.2 Graph Cannot Be Rendered** 1. The component logic graph cannot be displayed. 2. The application informs the Engineer that the graph could not be shown. 3. Any already-loaded trace is left unchanged. **UC-7.0.E.3 Event Stream Interrupted While Watching** 1. The stream breaks before the run reports completion. 2. The application does not present the partial trace as a finished validation result. 3. The application reports that the live update is incomplete.
Includes:	None
Priority:	High
Frequency of Use:	Performed during or after every design run. This is how the Engineer reads the result of UC-6 and any later repair in UC-8.
Business Rules:	1. No pass or fail verdict shown in the trace may be produced by the language model. 2. A value that is not published is reported as unchecked and is never replaced by a default. 3. Every verdict shown must carry the field name, the value read, and the source. 4. Watching the trace and the graph must not change saved design data.
Special Requirements:	1. Streamed events arrive in order with no gaps and no duplicates (DS-6). 2. A conflict displays every rule that ran on the failing part, not only the rule that failed (DS-15).
Assumptions:	The Engineer is looking at a run that belongs to them. The design workspace can render both the trace and the graph when the underlying data is available.
Notes and Issues:	This use case is the observation of results produced by UC-6 and updated by UC-8. It does not start a run and does not apply a repair. Whether an Engineer can hide the graph or the trace independently is unresolved. To be decided with the frontend developer before the interface contract is frozen.

Use Case ID:	UC-8		
Use Case Name:	Automatically Repair an Invalid Design		
Created By:	Samarth	Last Updated By:	
Date Created:	31-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows the system to automatically attempt to repair a board design when validation detects an issue. The model proposes an appropriate repair action from the supported repair actions, after which the engine applies the change and re-validates the design.
Trigger:	The validation engine detects that the current board design fails one or more validation checks.
Preconditions:	1. The Engineer has initiated a board-design run. 2. A board design has been generated. 3. The validation engine is available. 4. The generated design contains at least one validation issue requiring resolution.
Postconditions:	1. The proposed repair is applied to the board where possible. 2. The modified board is re-checked by the validation engine. 3. The latest board state and validation result are retained as part of the design run. 4. If the issue cannot be resolved automatically, the workflow may proceed to escalation.
Normal Flow:	**UC-8.0 Repair Invalid Design** 1. The validation engine identifies a validation failure in the current board design. 2. The system provides the detected issue and relevant design context to the model. 3. The model selects and proposes an appropriate repair from the supported repair actions. 4. The system applies the proposed repair to the board design. 5. The validation engine re-checks the modified board. 6. If the validation succeeds, the system continues the design workflow. 7. The updated validation trace and board state are made available to the Engineer.
Alternative Flows:	**UC-8.1 No Repair Required** 1. The board passes validation without requiring a repair. 2. The automatic repair process is skipped. 3. The system proceeds with the validated design. **UC-8.2 Further Repair Required** 1. The repaired board still fails validation. 2. The system evaluates the remaining issue. 3. Another supported repair may be proposed and the design is re-checked.

Exceptions:	**UC-8.0.E.1 Repair Cannot Be Applied** 1. The proposed repair cannot be successfully applied to the board. 2. The system retains the last valid board state where applicable. 3. The unresolved issue is recorded. **UC-8.0.E.2 Validation Engine Failure** 1. The validation engine cannot complete the re-check. 2. The system records the failure and does not treat the design as successfully validated.
Includes:	Board validation and design generation workflow
Priority:	High
Frequency of Use:	Performed whenever a generated or modified board fails validation during a design run.
Business Rules:	Only supported repair actions may be proposed and applied by the automated repair process. A repaired design must be re-validated before it can be treated as valid.
Special Requirements:	The system must preserve the validation and repair trace so that the Engineer can understand how the design changed.
Assumptions:	The model and validation engine are available during the design run.
Notes and Issues:	The model may propose only one of six repair actions defined by the engine: swap, change_topology, add_part, change_rail, relax_requirement, escalate. A slot the Engineer named explicitly is restricted to swap, change_rail and escalate, so a repair preserves the function that was asked for. A repaired design must be re-validated before it can be treated as valid.

Use Case ID:	UC-9		
Use Case Name:	Answer an Escalation		
Created By:	Samarth	Last Updated By:	
Date Created:	31-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an Engineer to provide additional information when the automated design workflow cannot safely determine how to proceed. The running workflow pauses using a LangGraph interrupt, presents a question to the Engineer, and waits for an answer before continuing. Relevant routes: POST /resume (body: thread_id, answer)
Trigger:	The design workflow reaches a situation that requires clarification or a decision from the Engineer.
Preconditions:	1. The Engineer has an active design run. 2. The workflow has reached an escalation requiring human input. 3. The run has been paused and the Engineer is presented with a question.
Postconditions:	1. The Engineer's response is recorded. 2. The paused workflow receives the supplied response. 3. The run is able to continue using the additional information. 4. The escalation and response remain associated with the relevant design run.
Normal Flow:	**UC-9.0 Answer Escalation** 1. The system determines that human input is required to continue the workflow. 2. A LangGraph interrupt pauses execution of the current run. 3. The application displays the escalation question to the Engineer. 4. The system waits for the Engineer's response. 5. The Engineer reviews the question and submits an answer. 6. The application sends the response back to the paused workflow. 7. The workflow resumes using the Engineer's response as additional context. 8. The design process continues.
Alternative Flows:	**UC-9.1 Engineer Does Not Answer Immediately** 1. The Engineer leaves the run without responding to the escalation. 2. The run remains paused. 3. The Engineer may return to the run later and provide the required response.

Exceptions:	**UC-9.0.E.1 Invalid or Missing Response** 1. The submitted response does not provide the information required by the workflow. 2. The system does not incorrectly continue the design process. 3. The Engineer is asked to provide the required information. **UC-9.0.E.2 Resume Fails** 1. The Engineer successfully submits an answer but the workflow cannot resume. 2. The system reports the failure while preserving the paused run and submitted response where possible.
Includes:	None
Priority:	High
Frequency of Use:	Performed whenever the automated design workflow encounters a situation requiring Engineer input.
Business Rules:	A workflow waiting for an escalation response must remain paused until the required Engineer input is supplied.
Special Requirements:	The paused workflow state must be preserved so that the Engineer can respond later without restarting the entire design process.
Assumptions:	The Engineer has access to the relevant project/run and sufficient information to answer the escalation question.
Notes and Issues:	The actual resume route can be added once confirmed.

Use Case ID:	UC-10		
Use Case Name:	Recover a Stopped Run		
Created By:	Lai Yi	Last Updated By:	
Date Created:	1-9-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an authenticated Engineer to continue a design run that stopped without finishing — because the client disconnected (abandoned) or the run failed (error). The system restarts the graph from its last completed checkpoint rather than from an empty board. Relevant routes: - POST /threads/{id}/continue — a run whose status is abandoned or error
Trigger:	The Engineer chooses to resume a paused design run, or the application sends the continue request after the information required to proceed has been supplied.
Preconditions:	1. The Engineer has successfully authenticated with the system. 2. A design run exists and is currently in a paused state. 3. The paused workflow state has been preserved and can be addressed by its thread id. 4. The run's status is abandoned or error
Postconditions:	1. The paused workflow receives the continue request. 2. If continuation succeeds, the run leaves the paused waiting state and proceeds from the interrupt. 3. The run is not restarted from an empty board. 4. If continuation fails, the run remains paused and is not recorded as complete.
Normal Flow:	**UC-10.0 Recover a Stopped Run** 1. A run ended in an abandoned or error state, for example because the connection dropped. 2. The Engineer chooses to continue it. 3. The client sends the request to POST /threads/{id}/continue. 4. The system restarts the graph from its last completed checkpoint. 5. Event sequence numbers continue from the stored high-water mark, so no frame is discarded by the client.
Alternative Flows:	**NA**

Exceptions:	**UC-10.0.E.1 Run Is Not in a Continuable State** POST /threads/{id}/continue accepts only runs whose status is abandoned or error, and rejects anything else with a conflict. A run paused at an interrupt is continued through POST /resume instead. **UC-10.0.E.2 Continue Request Fails** 1. POST /threads/{id}/continue cannot be completed. 2. The run remains paused. 3. The application reports the failure and does not treat the run as complete. **UC-10.0.E.3 Thread Cannot Be Found** 1. The system cannot find a thread for the given id that belongs to the Engineer. 2. No workflow is resumed. 3. The application informs the Engineer that the run could not be opened. **UC-10.0.E.4 Engineer Not Authenticated** 1. The system determines that the Engineer does not have valid authentication. 2. The continue request is rejected. 3. The paused run is left unchanged.
Includes:	None
Priority:	High
Frequency of Use:	Performed whenever a design run has been paused and the Engineer is ready to continue it, including after returning to a run later.
Business Rules:	1. A paused run must not be treated as complete until it has been continued and later finishes normally. 2. Resuming must continue from the preserved interrupt, not by discarding the run and starting POST /design again. 3. Only an authenticated Engineer who can access the thread may resume it.
Special Requirements:	The paused workflow state must be preserved so that the Engineer can resume later without restarting the entire design process.
Assumptions:	NA
Notes and Issues:	NA

Use Case ID:	UC-11		
Use Case Name:	View a Saved Board		
Created By:	Armaan Sinha	Last Updated By:	
Date Created:	1-9-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an Engineer to retrieve and view the board associated with an existing design thread. It enables the Engineer to return to a previously generated board and inspect its saved state without modifying or resuming the design run.
Trigger:	The Engineer selects an existing design thread and chooses to view its saved board.
Preconditions:	1. The Engineer is authenticated. 2. The selected design thread exists.
Postconditions:	1. The board associated with the selected thread is retrieved. 2. The saved board state is displayed to the Engineer. 3. Retrieving the board does not modify the board or resume the design run.
Normal Flow:	**UC-11.0 View a Saved Board** 1. The Engineer opens an existing design thread. 2. The Engineer requests the board associated with the thread. 3. The application requests the saved board using the thread identifier. 4. The system retrieves the board associated with the thread. 5. The application displays the saved board to the Engineer. 6. The Engineer reviews the board without changing the stored design state**.**
Alternative Flows:	**UC-11.1 No Board Available** 1. The selected thread exists but does not contain a saved board. 2. The application informs the Engineer that no board is currently available for the thread.
Exceptions:	**UC-11.0.E.1 Thread Not Found** 1. The requested thread does not exist or cannot be accessed by the Engineer. 2. No board is returned. 3. The application informs the Engineer that the requested thread or board could not be found. **UC-11.0.E.2 Board Retrieval Fails** 1. The system cannot retrieve the board because of an unexpected retrieval error. 2. The application informs the Engineer that the board could not be loaded and does not display incomplete data as the saved board.
Includes:	None
Priority:	High

Frequency of Use:	Performed whenever an Engineer returns to an existing design thread to inspect its board.
Business Rules:	1. An Engineer may only retrieve a board belonging to a thread they are authorised to access. 2. Viewing a saved board must not alter the board or the state of its design run.
Special Requirements:	1. The board displayed must accurately represent the saved board state returned for the selected thread.
Assumptions:	Board state is persisted and associated with its corresponding thread.
Notes and Issues:	1. Confirm whether /threads/{id}/board always returns the latest persisted board state. 2. Clarify the response when a thread exists but no persisted board is available. 3. Confirm that retrieving a board is strictly read-only and does not make the run active or resume it.

Use Case ID:	UC-12		
Use Case Name:	Validate an Existing BOM		
Created By:	Armaan Sinha	Last Updated By:	
Date Created:	1-9-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an Engineer to validate an existing Bill of Materials (BOM) independently of the board-design workflow. The Engineer provides a BOM either as a CSV file or as a list containing one Manufacturer Part Number (MPN) per line, and the system checks the supplied components and returns validation results.
Trigger:	The Engineer chooses to validate an existing BOM.
Preconditions:	1. The Engineer has access to the BOM validation feature. 2. The Engineer has an existing BOM to validate. 3. The BOM is provided in a supported input format.
Postconditions:	1. The submitted BOM is processed by the validation system. 2. Validation results are returned to the Engineer. 3. Components that cannot be validated are identified in the results where supported. 4. The submitted BOM itself is not modified by the validation operation.
Normal Flow:	**UC-12.0 Validate Existing BOM Using CSV** 1. The Engineer opens the BOM validation feature. 2. The Engineer provides an existing BOM as a CSV file. 3. The Engineer submits the BOM for validation. 4. The system reads and processes the submitted component information. 5. The system performs the available validation checks on the supplied parts. 6. The system collects the validation results and associated findings. 7. The application displays the results to the Engineer, including an indication when no validation issues are detected.
Alternative Flows:	**UC-12.1 Enter MPNs Manually** 1. Instead of uploading a CSV file, the Engineer enters or pastes one MPN per line. 2. The system processes the supplied MPN entries. 3. The system performs the same available validation checks on the supplied parts. 4. The application displays the validation results to the Engineer. **UC-12.2 One or More MPNs Cannot Be Resolved** 1. The system cannot identify or obtain sufficient information for one or more submitted MPNs. 2. The affected MPNs are identified in the validation results rather than being shown as successfully validated.

	3. Validation continues for the remaining MPNs where possible. 4. The application displays both the available validation results and the unresolved entries.
Exceptions:	**UC-12.0.E.1 Invalid Input Format** 1. The submitted BOM cannot be interpreted using a supported input format. 2. Validation is not started for the invalid input. 3. The Engineer is informed that the input must be corrected before validation can proceed. **UC-12.0.E.2 Validation Service Fails** 1. The system encounters an unexpected failure while validating the BOM. 2. The incomplete operation is not presented as a completed validation result. 3. The Engineer is informed that validation could not be completed.
Includes:	None
Priority:	High
Frequency of Use:	Performed whenever an Engineer wants to validate an existing BOM independently of generating a new board.
Business Rules:	1. Only supported BOM input formats may be processed. 2. A component that cannot be evaluated must not be represented as having successfully passed validation.
Special Requirements:	The system must support both CSV input and text input containing one MPN per line.
Assumptions:	Each submitted BOM entry is expected to provide an MPN or component identifier that the system can attempt to validate. Not every submitted MPN is guaranteed to be resolvable.
Notes and Issues:	1. The exact supported CSV structure, including required columns, must be confirmed and documented. 2. Confirm how duplicate MPNs are handled during validation. 3. Confirm whether findings produced by standalone BOM validation are retained and later visible through UC-15.

Use Case ID:	UC-13		
Use Case Name:	Export the BOM		
Created By:	Lai Yi	Last Updated By:	
Date Created:	1-9-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an authenticated Engineer to export the bill of materials of a design run as a CSV file. It does not generate the BOM (that is UC-6) and it does not validate an uploaded or external BOM (that is UC-12). The Engineer obtains a .csv file that reflects the BOM currently stored for the run. Relevant routes: ● the application's BOM export that returns a .csv file (exact path to be confirmed with the backend; Sparsh listed this as "Export the BOM.csv")
Trigger:	The Engineer chooses to export or download the bill of materials as a CSV file.
Preconditions:	1. The Engineer has successfully authenticated with the system. 2. A design run exists that the Engineer is allowed to open. 3. The run has a bill of materials that can be exported, or the application can determine that it does not. 4. The BOM export function is available.
Postconditions:	1. On success, the Engineer obtains a .csv file containing the bill of materials for that run. 2. The stored board and BOM are not modified by the export. 3. The CSV contents match the BOM held by the system for that run. 4. If export fails, the Engineer is not given an incomplete file presented as the official BOM.
Normal Flow:	**UC-13.0 Export the BOM as CSV** 1. The Engineer opens a run that has a bill of materials, from the design workspace or from the run view. 2. The application displays the current BOM. 3. The Engineer chooses the option to export or download the BOM as CSV. 4. The client requests the BOM in CSV form. 5. The system produces the CSV from the stored BOM for that run. 6. The application provides the .csv file for download. 7. The Engineer obtains the file and can open it outside the application.

Alternative Flows:	**UC-13.1 Export With Unresolved or Unchecked Items** 1. The BOM exists, but some slots are unresolved or some rules are unchecked. 2. The CSV still exports the current BOM. 3. Unresolved or unchecked items are represented honestly and are not written as if they had passed validation. **UC-13.2 No BOM Available Yet** 1. The run does not yet have a bill of materials. 2. No CSV download is completed. 3. The application informs the Engineer that there is nothing to export, instead of producing an empty file as if it were a valid BOM.
Exceptions:	**UC-13.0.E.1 Export Request Fails** 1. The system cannot produce or return the CSV. 2. The application informs the Engineer that the export failed. 3. No incomplete file is presented as the official BOM. **UC-13.0.E.2 Engineer Not Authenticated** 1. The system determines that the Engineer does not have valid authentication. 2. The export request is rejected. 3. No file is produced. **UC-13.0.E.3 File Cannot Be Downloaded** 1. The CSV is produced but the application cannot deliver it to the Engineer. 2. The application reports that the download failed. 3. The stored BOM is left unchanged.
Includes:	None
Priority:	Medium
Frequency of Use:	Performed after a run has a BOM, whenever the Engineer needs the parts list outside the application.
Business Rules:	1. The exported CSV must correspond to the BOM actually stored for that run. 2. Exporting the BOM must not modify the board, the run, or the stored BOM. 3. Only an authenticated Engineer who can access the run may export its BOM.
Special Requirements:	The downloaded file should be usable as CSV in a spreadsheet. Column layout is to be confirmed with the frontend and backend developers before the interface contract is frozen.
Assumptions:	A completed or in-progress run can expose the current BOM in a form that can be serialised to CSV.
Notes and Issues:	Exact export route is not in the truncated Telegram list beyond “Export the BOM.csv”. Confirm with Sparsh / the backend developer (possible forms include GET /threads/{id}/bom.csv). UC-12 (validate an existing BOM) is a different use case and must not be folded into this export.

Use Case ID:	UC-14		
Use Case Name:	Attach a Datasheet to Supply a Missing Value		
Created By:	Armaan Sinha	Last Updated By:	
Date Created:	1-9-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an Engineer to supply a manufacturer datasheet for a component whose thermal resistance (θJA) is not published by any catalogue source. The Engineer uploads the PDF together with the manufacturer part number and package; the system extracts the θJA figure and retains the quoted line it was read from, so that any verdict relying on it can cite its source.
Trigger:	A rule reports unchecked because a thermal figure is unavailable, or the Engineer chooses to attach a datasheet to a component.
Preconditions:	1. The Engineer has successfully authenticated with the system. 2. The Engineer has the datasheet PDF for the component. 3. The manufacturer part number and package are known.
Postconditions:	1. On success, the extracted figure is stored against that part with the verbatim line it was read from. 2. A rule that previously reported unchecked for that field can now evaluate and cite the datasheet. 3. If extraction fails, no figure is stored and no rule is allowed to treat the value as available.
Normal Flow:	**UC-14.0 Attach a Datasheet** 1. The Engineer selects a component whose thermal figure is unavailable. 2. The Engineer supplies the datasheet PDF, the manufacturer part number and the package. 3. The client sends the request to POST /datasheet. 4. The system validates that the part number, package and document are present and within the size limit. 5. The system extracts the θJA figure and the line it was read from. 6. The system stores the figure against that part. 7. The application confirms the value and shows the quoted line.
Alternative Flows:	**UC-14.1 Figure not found in the Document** The document is read but contains no usable θJA figure. No value is stored. The Engineer is told the figure could not be found, and the rule remains unchecked.

Exceptions:	**UC-14.0.E.1 Required Field Missing** The part number, package or document is empty. The system rejects the request (422) and stores nothing. **UC-14.0.E.2 Document Too Large** The PDF exceeds the size limit. The system rejects the request and informs the Engineer. **UC-14.0.E.3 Extraction Fails** The document cannot be parsed. No value is stored, and no partial figure is presented as extracted.
Includes:	None
Priority:	Medium
Frequency of Use:	Performed whenever an Engineer needs technical information about a component during design or validation.
Business Rules:	A figure may only be stored together with the line it was read from. A value that cannot be extracted must leave the rule reporting unchecked; it is never replaced by a default.
Special Requirements:	The lookup result must clearly identify the component to which the returned datasheet belongs.
Assumptions:	The system has access to a datasheet lookup source, but a matching datasheet is not guaranteed to exist for every component..
Notes and Issues:	1. Confirm the exact component identifier accepted by /datasheet, such as an MPN or internal component identifier. 2. Clarify how the system handles multiple possible datasheet matches or component variants. 3. Confirm whether /datasheet returns a datasheet file, URL/reference, extracted information, or a combination of these.

Use Case ID:	UC-15		
Use Case Name:	Browse Memory		
Created By:	Samarth	Last Updated By:	
Date Created:	31-8-2026	Date Last Updated:	

Actors:	Engineer
Description:	This use case allows an Engineer to browse information retained by the system from previous design activity. The Engineer can inspect remembered parts, identify the boards on which those parts appear, and view the findings recorded for them.
Triggers:	The Engineer opens the Application's memory interface
Preconditions:	1. The Engineer has successfully authenticated with the system. 2. The Engineer has access to the memory interface. 3. The system contains stored memory data from previous design activity.
Postconditions:	1. The requested memory information is displayed to the Engineer. 2. No board or memory entry is modified merely by browsing it. 3. The Engineer can identify relationships between stored parts, boards, and recorded findings.
Normal Flow:	**UC-15.0 Browse Memory** 1. The Engineer opens the memory interface. 2. The system retrieves the available stored part information. 3. The application displays the remembered parts to the Engineer. 4. The Engineer selects a part to inspect. 5. The system retrieves the stored information associated with the selected part. 6. The application displays the boards in which the part appears. 7. The application displays the recorded findings associated with the part. 8. The Engineer browses the information as required.
Alternative Flows:	**UC-15.1 No Stored Memory** 1. The Engineer opens the memory interface before any relevant information has been recorded. 2. The application displays an empty-state message. 3. No error is generated. **UC-15.2 Part Has No Recorded Findings** 1. The Engineer selects a stored part for which no findings have been recorded. 2. The application displays the available part/board information and indicates that no findings are available.
Exceptions:	**UC-15.0.E.1 Memory Retrieval Fails** 1. The system cannot retrieve the requested memory information.

	2. The application informs the Engineer that the information could not be loaded. 3. No stored memory information is modified or deleted.
Includes:	None
Priority:	Medium
Frequency of Use:	Performed whenever an Engineer wants to review information learned or recorded during previous board-design activity.
Business Rules:	Memory information displayed to an Engineer must correspond to information actually recorded by the system. Browsing memory must not modify existing records.
Special Requirements:	Relationships between parts, the boards containing them, and their recorded findings should be clearly visible to the Engineer.
Assumptions:	Relevant memory records are retained and accessible by the application.
Notes and Issues:	None